ЛАБОРАТОРНА РОБОТА № 3
Поведінкові шаблони

Мета роботи: навчитися реалізовувати структурні шаблони проєктування Ланцюжок відповідальностей, Посередник, Спостерігач, Стратегія, Мементо

Завдання 1: Ланцюжок відповідальностей.

1. Створіть систему підтримки користувачів.
2. Вона має функціонувати так, як покрокове меню (наприклад, коли телефонуєте до оператора мобільного зв'язку). Але замість голосових повідомлень Ви маєте виводити відповідні повідомлення в консоль і чекати на вибір користувача.
3. Система повинна мати мінімум 4 рівні. Всі питання мають на меті обрати правильний рівень підтримки (тобто Handler) для користувача. Тобто вже на першому питанні може бути підібрано правильний рівень підтримки, тоді меню закінчується. А може бути так, що на жодному питанні не буде знайдено правильний рівень (Handler), тоді меню має повторитися.
4. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг програми:

```
using System;

namespace Lab4_Chain
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;


            Handler level4 = new TechExpertHandler();
            Handler level3 = new SeniorSupportHandler(level4);
            Handler level2 = new JuniorSupportHandler(level3);
            Handler start = new ReceptionHandler(level2);

            bool solved = false;
            while (!solved)
            {
                Console.WriteLine("\nСистема підтримки користувачів ");
                solved = start.HandleRequest();
                if (!solved) Console.WriteLine("На жаль, ми не змогли допомогти.
Спробуйте ще раз.");
            }
            Console.ReadKey();
        }
    }

    public abstract class Handler
    {
        protected Handler _next;
        public Handler(Handler next = null) => _next = next;
        public abstract bool HandleRequest();
    }
```

```
    public class ReceptionHandler : Handler
    {
        public ReceptionHandler(Handler next) : base(next) { }
        public override bool HandleRequest()
        {
            Console.WriteLine("1. У вас загальне питання по сервісу? (так/ні)");
            if (Console.ReadLine().ToLower() == "так")
            {
                Console.WriteLine("Відповідь: Наш сервіс працює 24/7. Дякуємо!");
                return true;
            }
            return _next?.HandleRequest() ?? false;
        }
    }


    public class JuniorSupportHandler : Handler
    {
        public JuniorSupportHandler(Handler next) : base(next) { }
        public override bool HandleRequest()
        {
            Console.WriteLine("2. У вас проблема з входом в акаунт? (так/ні)");
            if (Console.ReadLine().ToLower() == "так")
            {
                Console.WriteLine("Відповідь: Натисніть 'Забули пароль' на формі
входу.");
                return true;
            }
            return _next?.HandleRequest() ?? false;
        }
    }


    public class SeniorSupportHandler : Handler
    {
        public SeniorSupportHandler(Handler next) : base(next) { }
        public override bool HandleRequest()
        {
            Console.WriteLine("3. У вас помилка при оплаті замовлення? (так/ні)");
            if (Console.ReadLine().ToLower() == "так")
            {
                Console.WriteLine("Відповідь: Перевірте ліміт вашої картки або зверніться
в банк.");
                return true;
            }
            return _next?.HandleRequest() ?? false;
        }
    }


    public class TechExpertHandler : Handler
    {
        public override bool HandleRequest()
        {
            Console.WriteLine("4. У вас технічний баг у коді програми? (так/ні)");
            if (Console.ReadLine().ToLower() == "так")
            {
                Console.WriteLine("Відповідь: Запит передано розробникам. Очікуйте на
пошту.");
                return true;
            }
            return false;
        }
    }
}
```

```
Консоль отладки Microsoft Vi

Система підтримки користувачів
1. У вас загальне питання по сервісу? (так/ні)
так
Відповідь: Наш сервіс працює 24/7. Дякуємо!
В
D:\Lab\kpz_lab4.1\kpz_lab4.1\bin\Debug\net10.0\kpz_lab4.1.exe (процесс 7872) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Ав
томатически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно…
```

Рис.1.Результат виконання програми:

```
D:\Lab\kpz_lab4.1\kpz_lab4.1\l

Система підтримки користувачів
1. У вас загальне питання по сервісу? (так/ні)
ні
2. У вас проблема з входом в акаунт? (так/ні)
ні
3. У вас помилка при оплаті замовлення? (так/ні)
ні
4. У вас технічний баг у коді програми? (так/ні)
ні
На жаль, ми не змогли допомогти. Спробуйте ще раз.
```

Рис.2.Результат виконання програми:

Завдання 2: Посередник.

1. Відрефакторте код продемонстрований на лекції за допомогою використання шаблону Посередник.
2. В результаті рефакторингу Aircraft не повинен “знати” про Runway і навпаки. Обидві сутності повинні “знати” лише про CommandCentre.
3. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг програми:

```
using System;
using System.Collections.Generic;

namespace Lab4_Mediator
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;


            var commandCentre = new CommandCentre();


            var runway1 = new Runway(commandCentre);
            var aircraft1 = new Aircraft("Boeing 747", commandCentre);
            var aircraft2 = new Aircraft("Airbus A320", commandCentre);

            Console.WriteLine("Смуга зайнята ");
            runway1.IsFree = false; // Смуга зайнята
            aircraft1.RequestLanding();

            Console.WriteLine("\nСмуга вільна ");
```

```
            runway1.IsFree = true;
            aircraft1.RequestLanding();

            Console.WriteLine("\n Другий літак");
            aircraft2.RequestLanding();

            Console.ReadKey();
        }
    }

    public interface IMediator
    {
        void RegisterAircraft(Aircraft aircraft);
        void RegisterRunway(Runway runway);
        bool CanLand(Aircraft aircraft);
    }


    public class CommandCentre : IMediator
    {
        private List<Aircraft> _aircrafts = new List<Aircraft>();
        private List<Runway> _runways = new List<Runway>();

        public void RegisterAircraft(Aircraft aircraft) => _aircrafts.Add(aircraft);
        public void RegisterRunway(Runway runway) => _runways.Add(runway);

        public bool CanLand(Aircraft aircraft)
        {

            foreach (var runway in _runways)
            {
                if (runway.IsFree) return true;
            }
            return false;
        }
    }


    public class Aircraft
    {
        private IMediator _mediator;
        public string Name { get; }

        public Aircraft(string name, IMediator mediator)
        {
            Name = name;
            _mediator = mediator;
            _mediator.RegisterAircraft(this);
        }

        public void RequestLanding()
        {
            Console.WriteLine($"{Name}: Запитую дозвіл на посадку у диспетчера...");
            if (_mediator.CanLand(this))
            {
                Console.WriteLine($"Диспетчер: Дозвіл для {Name} НАДАНО.");
            }
            else
            {
                Console.WriteLine($"Диспетчер: Дозвіл для {Name} ВІДХИЛЕНО. Смуги
зайняті.");
            }
        }
    }
```

```
    public class Runway
    {
        private IMediator _mediator;
        public bool IsFree { get; set; }

        public Runway(IMediator mediator)
        {
            _mediator = mediator;
            _mediator.RegisterRunway(this);
        }
    }
}
```

```
D:\Lab\kpz_lab.4.2\kpz_lab.4.2

Смуга зайнята
Boeing 747: Запитую дозвіл на посадку у диспетчера...
Диспетчер: Дозвіл для Boeing 747 ВІДХИЛЕНО. Смуги зайняті.

Смуга вільна
Boeing 747: Запитую дозвіл на посадку у диспетчера...
Диспетчер: Дозвіл для Boeing 747 НАДАНО.

 Другий літак
Airbus A320: Запитую дозвіл на посадку у диспетчера...
Диспетчер: Дозвіл для Airbus A320 НАДАНО.
```

Рис.3.Результат виконання програми:

Завдання 3: Спостерігач.

1. Додайте до Вашого LightHTML з завдання 5 ЛР №3 можливість додавання EventListener до Ваших HTML елементів.
2. Додайте можливість підписки не різні івенти (“click”, “mouseover” тощо).
3. Покажіть правильність роботи свого коду запустивши його в головному методі програми.
4. Це завдання виконайте в окремому PR.

Лістинг програми:

```
using System;
using System.Collections.Generic;

namespace Lab4_Observer
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;


            var button = new LightElementNode("button", "inline", "double");


            button.AddEventListener("click", (eventName) => {
                Console.WriteLine("Логгер: Виявлено натискання на елемент.");
            });

            button.AddEventListener("click", (eventName) => {
                Console.WriteLine("Аналітика: Відправлено дані про клік на сервер.");
            });


            button.AddEventListener("mouseover", (eventName) => {
```

```
                Console.WriteLine("Інтерфейс: Колір елемента змінено при наведенні.");
            });


            Console.WriteLine("Симуляція натискання");
            button.TriggerEvent("click");

            Console.WriteLine("\nСимуляція наведення ");
            button.TriggerEvent("mouseover");

            Console.ReadKey();
        }
    }

    public abstract class LightNode
    {
        public abstract string OuterHTML();
    }

    public class LightElementNode : LightNode
    {
        private string _tagName;
        private string _displayType;
        private string _closingType;


        private Dictionary<string, List<Action<string>>> _eventListeners = new Diction-
ary<string, List<Action<string>>>();

        public LightElementNode(string tag, string display, string closing)
        {
            _tagName = tag;
            _displayType = display;
            _closingType = closing;
        }


        public void AddEventListener(string eventType, Action<string> handler)
        {
            if (!_eventListeners.ContainsKey(eventType))
            {
                _eventListeners[eventType] = new List<Action<string>>();
            }
            _eventListeners[eventType].Add(handler);
        }


        public void TriggerEvent(string eventType)
        {
            if (_eventListeners.ContainsKey(eventType))
            {
                foreach (var handler in _eventListeners[eventType])
                {
                    handler.Invoke(eventType);
                }
            }
        }

        public override string OuterHTML() => $"<{_tagName}></{_tagName}>";
    }
}
```

```
Консоль отладки Microsoft Vi
Симуляція натискання
Логгер: Виявлено натискання на елемент.
Аналітика: Відправлено дані про клік на сервер.

Симуляція наведення
Інтерфейс: Колір елемента змінено при наведенні.
```

Рис.4.Результат виконання програми:

Завдання 4: Стратегія.

1. Додайте до Вашого LightHTML з завдання 5 ЛР №3 новий елемент Image, який за допомогою стратегії в залежності від переданого href буде завантажувати картинку або з файлової системи або з мережі.
2. Покажіть правильність роботи свого коду запустивши його в головному методі програми.
3. Це завдання виконайте в окремому PR

Лістинг програми:

```
using System;
using System.Text;

namespace Lab4_Strategy
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = Encoding.UTF8;


            var networkImg = new LightImage("https://dovidka.biz.ua/zagadka-pro-sobaku-
dlya-ditey/", new NetworkLoadStrategy());
            networkImg.Load();

            var localImg = new LightImage("C:/Users/Admin/Pictures/dog_photo.jpg", new
FileSystemLoadStrategy());
            localImg.Load();

            Console.ReadKey();
        }
    }

    public interface IImageLoadStrategy
    {
        void Load(string href);
    }

    public class NetworkLoadStrategy : IImageLoadStrategy
    {
        public void Load(string href)
        {
            Console.WriteLine($"[Мережа] Завантаження зображення за URL-адресою:
{href}");
        }
    }

    public class FileSystemLoadStrategy : IImageLoadStrategy
    {
```

```
        public void Load(string href)
        {
            Console.WriteLine($"[Файлова система] Читання файлу з локального диска:
{href}");
        }
    }

    public abstract class LightNode
    {
        public abstract string OuterHTML();
    }

    public class LightImage : LightNode
    {
        private string _href;
        private IImageLoadStrategy _loadStrategy;

        public LightImage(string href, IImageLoadStrategy strategy)
        {
            _href = href;
            _loadStrategy = strategy;
        }

        public void Load()
        {
            _loadStrategy.Load(_href);
        }

        public override string OuterHTML()
        {
            return $"<img src=\"{_href}\" />";
        }
    }
}
```

```
D:\Lab\kpz_lab4.4\kpz_lab4.4\

[Мережа] Завантаження зображення за URL-адресою: https://dovidka.biz.ua/zagadka-pro-sobaku-dlya-ditey/
[Файлова система] Читання файлу з локального диска: C:/Users/Admin/Pictures/dog_photo.jpg
```

Рис.5.Результат виконання програми:

Завдання 5: Мемento.

1. Уявіть, що ви створюєте програму текстового редактора.
2. Для цього завдання буде достатньо POC-версії: клас TextEditor, який містить в собі поточну версію текстового документу, який представлений спеціальним класом TextDocument. Назви класам можете давати власні.
3. Реалізуйте функцію збереження і скасування, яка дозволить користувачам відмінити зміни, зроблені в документі. Завдання полягає в тому, щоб зберегти стан документа в різні моменти часу та відновити його, коли це необхідно, не розкриваючи внутрішню реалізацію документа чи текстового редактору.

Лістинг програми:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Lab4_Memento
{
    class Program
    {
        static void Main(string[] args)
        {
```

```
        Console.OutputEncoding = Encoding.UTF8;

        var editor = new TextEditor();
        var history = new EditorHistory();

        editor.SetText("Початкови текст.");
        history.Push(editor.Save());

        editor.SetText("Додано новий рядок.");
        history.Push(editor.Save());

        editor.SetText("Останні правки.");

        Console.WriteLine("Поточний стан ");
        editor.Print();

        Console.WriteLine("\nСкасування (Undo 1) ");
        editor.Restore(history.Pop());
        editor.Print();

        Console.WriteLine("\nСкасування (Undo 2)");
        editor.Restore(history.Pop());
        editor.Print();

        Console.ReadKey();
    }
}

public class TextDocument
{
    public string Content { get; }
    public DateTime SaveTime { get; }

    public TextDocument(string content)
    {
        Content = content;
        SaveTime = DateTime.Now;
    }
}

public class TextEditor
{
    private string _textContent;

    public void SetText(string text) => _textContent = text;

    public TextDocument Save()
    {
        return new TextDocument(_textContent);
    }

    public void Restore(TextDocument memento)
    {
        if (memento != null)
        {
            _textContent = memento.Content;
        }
    }

    public void Print() => Console.WriteLine($"Вміст: {_textContent}");
}

public class EditorHistory
{
    private Stack<TextDocument> _history = new Stack<TextDocument>();
```

```
        public void Push(TextDocument memento) => _history.Push(memento);

        public TextDocument Pop()
        {
            if (_history.Count > 0) return _history.Pop();
            return null;
        }
    }
}
```

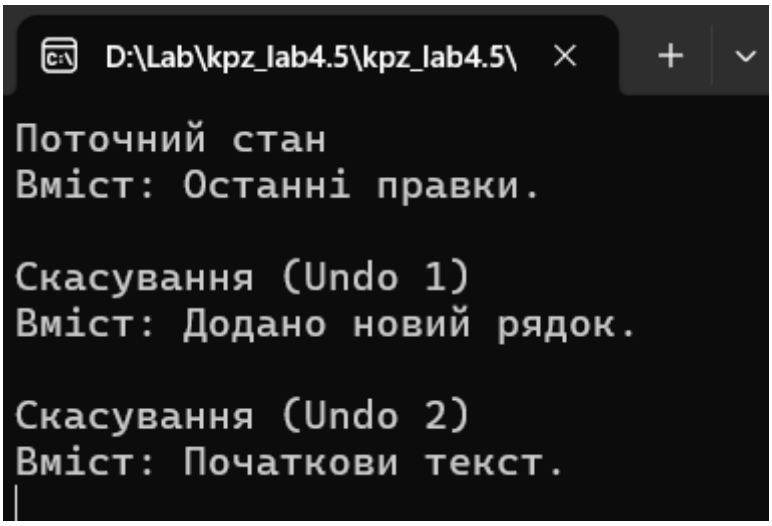

Рис.6.Результат виконання програми